SIMATS ENGINEERING

ASSESSMENT TOOL 1 -Constraint Based Problem Solving

COURSE NAME: R PROGRAMMING **COURSE CODE: ITA0402**

COURSE OUTCOME COVERED

CO1: Apply R programming fundamentals including data structures, vectors, matrices, arrays, and class types to perform mathematical operations and manage data effectively. Identify and implement appropriate control structures to solve a particular programming problem
(Bloom Level -BL3- BL4)

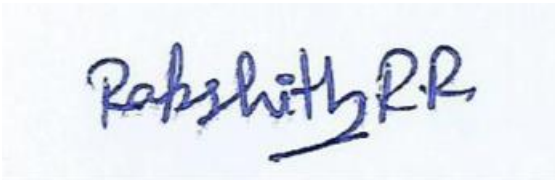
Assessment Tool Weightage: 20%

QUESTIONS: 5 Questions **Total Marks: 100**

Code of Conduct:

*I **RAKSHITH R.R (192324156)** ((Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



1. Student Performance Analysis

A college stores the marks of 10 students in Mathematics, Physics, and Chemistry. Write an R program to:

- Store the marks in a matrix.
- Calculate subject-wise averages.
- Find the highest and lowest scorer in each subject.
- Display students scoring above the overall average.

2. Employee Salary Processing

An organization maintains employee details including Employee ID, Name, Basic Pay, and Allowance.
Write an R program using appropriate data structures to:

- Calculate Gross Salary.
- Identify employees earning above ₹50,000.
- Display the employee with the highest salary.
- Generate a summary report.

3. Weather Data Analysis Using Arrays

Daily temperature data for three cities is collected for 7 days.
Write an R program to:

- Store the data in a 3-dimensional array.
- Find the average temperature for each city.
- Identify the hottest day.
- Compare city-wise temperature variations.

4. Product Inventory Management

A store maintains product information such as Product ID, Product Name, Quantity, and Price. Using lists and vectors in R:

- Calculate inventory value for each product.
- Find products with stock below a threshold value.
- Generate a stock summary report.

5. Bank Account Processing Using Functions

Develop an R program with user-defined functions to:

- Deposit money.

- Withdraw money.
- Check account balance.
- Display transaction history.
  Use suitable variables and functions.

6. Hospital Patient Data Analysis

A hospital stores patient details including Age, Gender, and Treatment Cost.
Write an R program to:

- Store the information in a data frame.
- Calculate average treatment cost.
- Find patients above 60 years of age.
- Generate statistical summaries.

7. Logical Vector-Based Voting Eligibility System

Create a dataset containing Name and Age of citizens.
Using logical vectors:

- Determine voting eligibility (Age ≥ 18).
- Count eligible and ineligible voters.
- Display the percentage of eligible voters.

8. Matrix-Based Sales Analysis

A company records monthly sales for four products across three regions.
Write an R program to:

- Store the sales data in a matrix.
- Calculate region-wise and product-wise sales totals.
- Identify the best-performing product.
- Display the region with maximum sales.

9. Library Management Using S3 Class

Design an S3 class named **Book** with attributes:

- Book ID
- Title
- Author
- Number of Copies

Write methods to:

- Display book details.
- Update available copies.
- Search for a specific book.

10. University Information System Using S4 Class and Reference Class

Create an S4 class named **Student** with slots:

- Roll Number
- Name
- CGPA

Create a Reference Class to:

- Add student records.
- Modify CGPA.
- Display student details.
- Find students with CGPA greater than 8.0.

RUBRICS FOR CALCULATION:

Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

1. Student Performance Analysis

A college stores the marks of 10 students in Mathematics, Physics, and Chemistry. Write an R program to:

- Store the marks in a matrix.
- Calculate subject-wise averages.
- Find the highest and lowest scorer in each subject.
- Display students scoring above the overall average.

Approach:

A 10 x 3 matrix (students x subjects) is created with named rows and columns. colMeans() gives subject-wise averages; a for loop with which.max()/which.min() finds the highest and lowest scorer in each subject; rowMeans() is compared with the overall mean() to list students scoring above the overall average.

Code:

```
# Q1: Student Performance Analysis (Matrix)
marks <- matrix(c(78,85,72, 92,88,95, 65,70,68, 81,79,84, 55,62,58,
                  90,94,89, 73,68,77, 86,91,82, 60,58,64, 95,97,93),
                nrow = 10, ncol = 3, byrow = TRUE,
                dimnames = list(paste0("Student", 1:10),
                                c("Maths", "Physics", "Chemistry")))
cat("Marks Matrix:\n"); print(marks)


cat("\nSubject-wise Averages:\n"); print(colMeans(marks))


cat("\nHighest and Lowest Scorer in Each Subject:\n")
for (sub in colnames(marks)) {
  hi <- rownames(marks)[which.max(marks[, sub])]
  lo <- rownames(marks)[which.min(marks[, sub])]
  cat(sub, "-> Highest:", hi, "(", max(marks[, sub]), ")",
      "| Lowest:", lo, "(", min(marks[, sub]), ")\n")
}


overall_avg <- mean(marks)
cat("\nOverall Average:", round(overall_avg, 2), "\n")
```

```
student_avg <- rowMeans(marks)
cat("Students scoring above overall average:\n")
print(round(student_avg[student_avg > overall_avg], 2))
```

Output:

Marks Matrix:

	Maths	Physics	Chemistry
Student1	78	85	72
Student2	92	88	95
Student3	65	70	68
Student4	81	79	84
Student5	55	62	58
Student6	90	94	89
Student7	73	68	77
Student8	86	91	82
Student9	60	58	64
Student10	95	97	93

Subject-wise Averages:

Maths	Physics	Chemistry
77.5	79.2	78.2

Highest and Lowest Scorer in Each Subject:

Maths -> Highest: Student10 (95) | Lowest: Student5 (55)

Physics -> Highest: Student10 (97) | Lowest: Student9 (58)

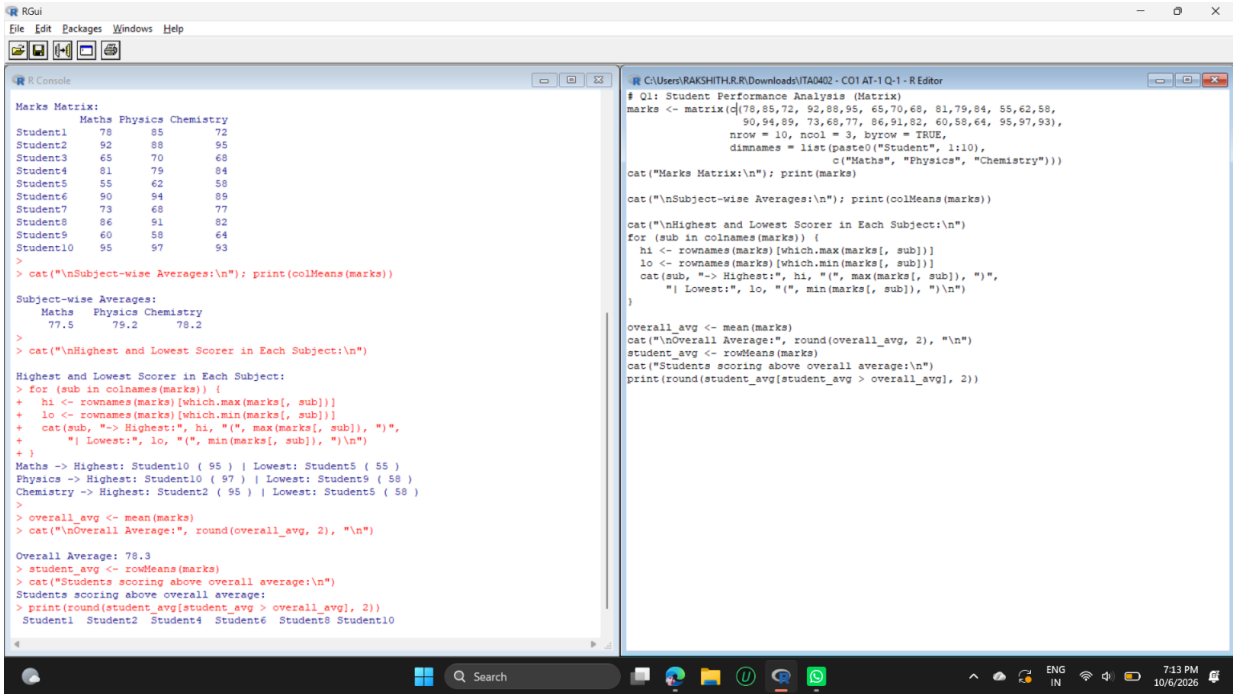
Chemistry -> Highest: Student2 (95) | Lowest: Student5 (58)

Overall Average: 78.3

Students scoring above overall average:

Student1	Student2	Student4	Student6	Student8	Student10
78.33	91.67	81.33	91.00	86.33	95.00

Output Screenshot:



2. Employee Salary Processing

An organization maintains employee details including Employee ID, Name, Basic Pay, and Allowance.
Write an R program using appropriate data structures to:

- Calculate Gross Salary.
- Identify employees earning above ₹50,000.
- Display the employee with the highest salary.
- Generate a summary report.

Approach:

Employee details are stored in a data frame (suitable for mixed data types). Gross Salary = Basic Pay + Allowance is added as a new column. Logical indexing filters employees above Rs.50,000, which.max() finds the top earner, and cat() with sum/mean/max/min produces the summary report.

Code:

```
# Q2: Employee Salary Processing (Data Frame)
emp <- data.frame(
  EmpID     = 101:108,
  Name      = c("Arun","Priya","Kumar","Divya","Rahul","Sneha","Vikram","Meena"),
  BasicPay  = c(30000, 45000, 28000, 52000, 38000, 60000, 25000, 41000),
  Allowance = c(8000, 12000, 6000, 15000, 9000, 18000, 5000, 10000)
)
emp$GrossSalary <- emp$BasicPay + emp$Allowance
cat("Employee Data with Gross Salary:\n"); print(emp)


cat("\nEmployees earning above Rs.50,000:\n")
print(emp[emp$GrossSalary > 50000, c("EmpID","Name","GrossSalary")])


cat("\nEmployee with Highest Salary:\n")
print(emp[which.max(emp$GrossSalary), c("EmpID","Name","GrossSalary")])


cat("\n----- SUMMARY REPORT -----\n")
cat("Total Employees      :", nrow(emp), "\n")
cat("Total Salary Payout  :", sum(emp$GrossSalary), "\n")
```

```
cat("Average Gross Salary :", mean(emp$GrossSalary), "\n")
cat("Highest Gross Salary :", max(emp$GrossSalary), "\n")
cat("Lowest Gross Salary  :", min(emp$GrossSalary), "\n")
cat("Employees above 50000:", sum(emp$GrossSalary > 50000), "\n")
```

Output:

Employee Data with Gross Salary:

	EmpID	Name	BasicPay	Allowance	GrossSalary
1	101	Arun	30000	8000	38000
2	102	Priya	45000	12000	57000
3	103	Kumar	28000	6000	34000
4	104	Divya	52000	15000	67000
5	105	Rahul	38000	9000	47000
6	106	Sneha	60000	18000	78000
7	107	Vikram	25000	5000	30000
8	108	Meena	41000	10000	51000

Employees earning above Rs.50,000:

	EmpID	Name	GrossSalary
2	102	Priya	57000
4	104	Divya	67000
6	106	Sneha	78000
8	108	Meena	51000

Employee with Highest Salary:

	EmpID	Name	GrossSalary
6	106	Sneha	78000

----- SUMMARY REPORT -----

Total Employees : 8

Total Salary Payout : 402000

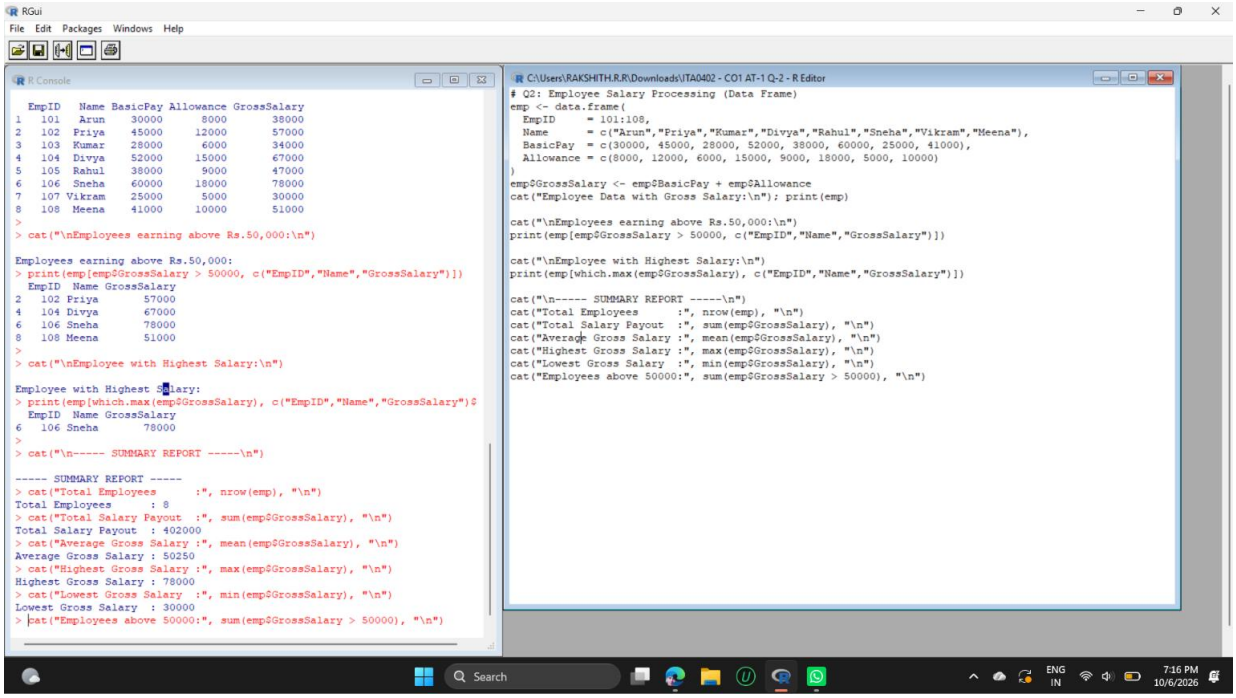
Average Gross Salary : 50250

Highest Gross Salary : 78000

Lowest Gross Salary : 30000

Employees above 50000: 4

Output Screenshot:



6. Hospital Patient Data Analysis

A hospital stores patient details including Age, Gender, and Treatment Cost. Write an R program to:

- Store the information in a data frame.
- Calculate average treatment cost.
- Find patients above 60 years of age.
- Generate statistical summaries.

Approach:

Patient records are stored in a data frame. mean() gives the average treatment cost, logical indexing (Age > 60) selects senior patients, summary() generates the statistical summary, and tapply()/sd() give the gender-wise average cost and standard deviation.

Code:

```
# Q6: Hospital Patient Data Analysis (Data Frame)
patients <- data.frame(
  PatientID     = 1:10,
  Age           = c(25, 67, 45, 72, 34, 61, 58, 80, 29, 64),
  Gender        = c("M","F","F","M","F","M","F","M","F","M"),
  TreatmentCost = c(12000, 45000, 20000, 60000, 15000, 38000,
                    25000, 75000, 11000, 42000)
)
cat("Patient Data:\n"); print(patients)


cat("\nAverage Treatment Cost:", mean(patients$TreatmentCost), "\n")


cat("\nPatients above 60 years:\n")
print(patients[patients$Age > 60, ])


cat("\nStatistical Summary:\n"); print(summary(patients))


cat("\nAverage Cost by Gender:\n")
print(tapply(patients$TreatmentCost, patients$Gender, mean))
cat("\nStd Deviation of Cost:", round(sd(patients$TreatmentCost), 2), "\n")
```

Output:

Patient Data:

	PatientID	Age	Gender	TreatmentCost
1	1	25	M	12000
2	2	67	F	45000
3	3	45	F	20000
4	4	72	M	60000
5	5	34	F	15000
6	6	61	M	38000
7	7	58	F	25000
8	8	80	M	75000
9	9	29	F	11000
10	10	64	M	42000

Average Treatment Cost: 34300

Patients above 60 years:

	PatientID	Age	Gender	TreatmentCost
2	2	67	F	45000
4	4	72	M	60000
6	6	61	M	38000
8	8	80	M	75000
10	10	64	M	42000

Statistical Summary:

PatientID	Age	Gender	TreatmentCost
Min. : 1.00	Min. :25.00	Length :10	Min. :11000
1st Qu.: 3.25	1st Qu.:36.75	N.unique : 2	1st Qu.:16250
Median : 5.50	Median :59.50	N.blank : 0	Median :31500
Mean : 5.50	Mean :53.50	Min.nchar: 1	Mean :34300
3rd Qu.: 7.75	3rd Qu.:66.25	Max.nchar: 1	3rd Qu.:44250
Max. :10.00	Max. :80.00		Max. :75000

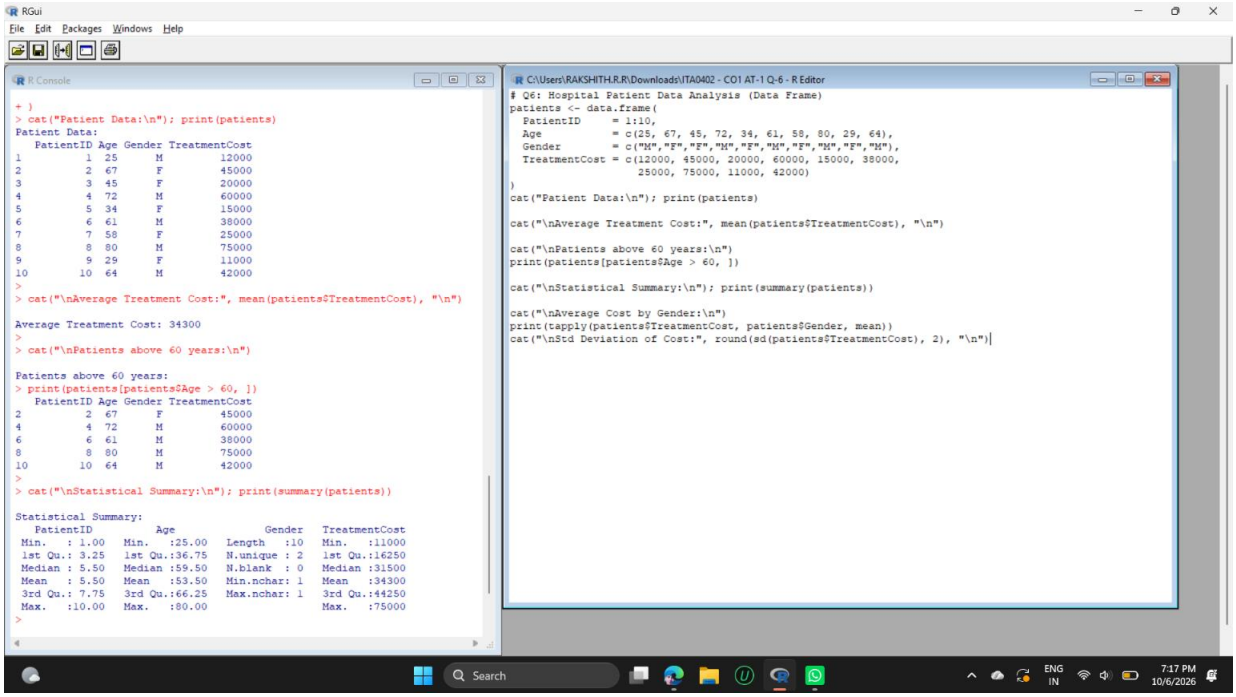
Average Cost by Gender:

F	M

23200 45400

Std Deviation of Cost: 21623.29

Output Screenshot:



7. Logical Vector-Based Voting Eligibility System

Create a dataset containing Name and Age of citizens. Using logical vectors:

- Determine voting eligibility (Age ≥ 18).
- Count eligible and ineligible voters.
- Display the percentage of eligible voters.

Approach:

Name and Age are stored as vectors/data frame. The logical vector Age >= 18 marks eligibility; since TRUE counts as 1, sum(eligible) counts eligible voters and sum(!eligible) counts ineligible ones. The percentage is (eligible / total) x 100.

Code:

```
# Q7: Voting Eligibility System (Logical Vectors)
name <- c("Arun","Priya","Kumar","Divya","Rahul","Sneha","Vikram","Meena","Ravi","Anita")
age  <- c(25, 17, 18, 16, 45, 30, 15, 22, 17, 60)
citizens <- data.frame(Name = name, Age = age)


eligible <- citizens$Age >= 18
citizens$Eligible <- eligible
cat("Voting Eligibility Table:\n"); print(citizens)


cat("\nEligible Voters  :", sum(eligible), "\n")
cat("Ineligible Voters:", sum(!eligible), "\n")
pct <- (sum(eligible) / length(eligible)) * 100
cat("Percentage of Eligible Voters:", pct, "%\n")


cat("\nEligible Names:", citizens$Name[eligible], "\n")
```

Output:

```
Voting Eligibility Table:
    Name Age Eligible
1   Arun  25     TRUE
2  Priya  17    FALSE
```

3 Kumar 18 TRUE

4 Divya 16 FALSE

5 Rahul 45 TRUE

6 Sneha 30 TRUE

7 Vikram 15 FALSE

8 Meena 22 TRUE

9 Ravi 17 FALSE

10 Anita 60 TRUE

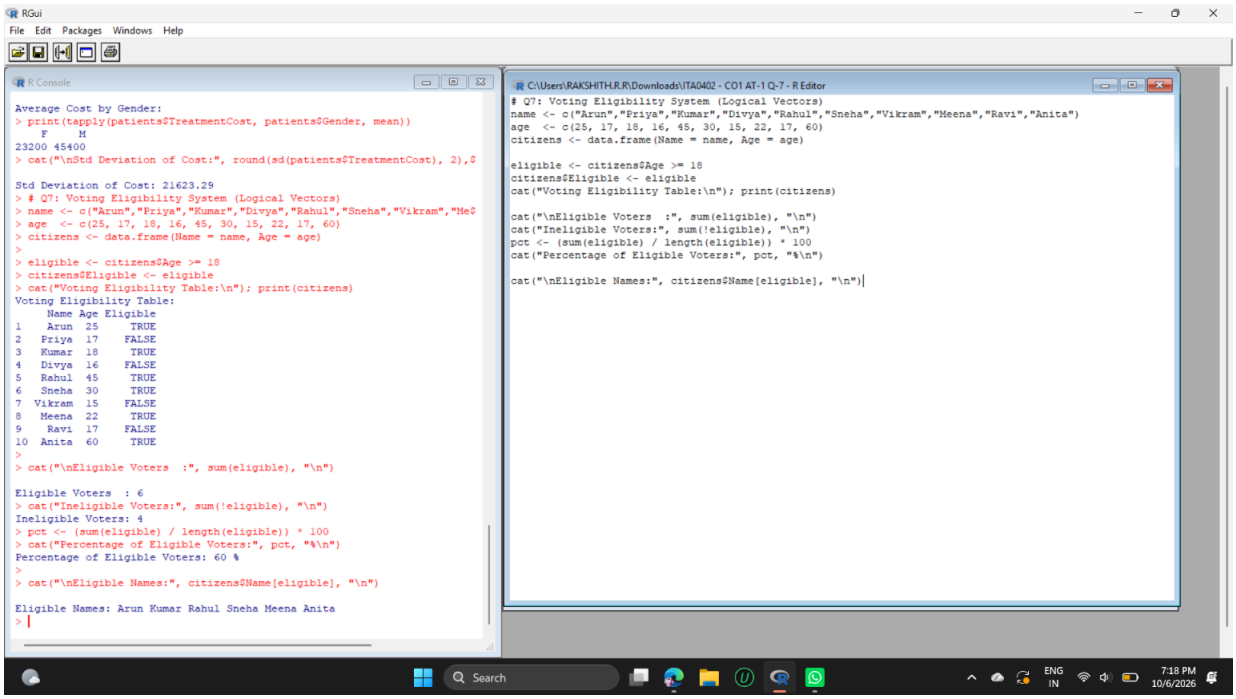
Eligible Voters : 6

Ineligible Voters: 4

Percentage of Eligible Voters: 60 %

Eligible Names: Arun Kumar Rahul Sneha Meena Anita

Output Screenshot:



8. Matrix-Based Sales Analysis

A company records monthly sales for four products across three regions. Write an R program to:

- Store the sales data in a matrix.
- Calculate region-wise and product-wise sales totals.
- Identify the best-performing product.
- Display the region with maximum sales.

Approach:

Monthly sales are stored in a 4 x 3 matrix (products x regions). rowSums() gives product-wise totals and colSums() gives region-wise totals; names(which.max()) identifies the best-performing product and the region with maximum sales.

Code:

```
# Q8: Matrix-Based Sales Analysis
sales <- matrix(c(25000, 30000, 28000,
                  40000, 35000, 45000,
                  15000, 20000, 18000,
                  32000, 28000, 36000),
                nrow = 4, ncol = 3, byrow = TRUE,
                dimnames = list(c("Product A","Product B","Product C","Product D"),
                                c("North","South","East")))
cat("Sales Matrix:\n"); print(sales)


product_total <- rowSums(sales)
region_total  <- colSums(sales)
cat("\nProduct-wise Total Sales:\n"); print(product_total)
cat("\nRegion-wise Total Sales:\n");  print(region_total)


best_product <- names(which.max(product_total))
cat("\nBest-Performing Product:", best_product,
    "with sales of", max(product_total), "\n")


best_region <- names(which.max(region_total))
```

```
cat("Region with Maximum Sales:", best_region,
    "with sales of", max(region_total), "\n")
```

Output:

Sales Matrix:

```
          North South  East
Product A 25000 30000 28000
Product B 40000 35000 45000
Product C 15000 20000 18000
Product D 32000 28000 36000
```

Product-wise Total Sales:

```
Product A Product B Product C Product D
    83000    120000     53000     96000
```

Region-wise Total Sales:

```
 North  South   East
112000 113000 127000
```

Best-Performing Product: Product B with sales of 120000

Region with Maximum Sales: East with sales of 127000

Output Screenshot:

